

Getting the Maximum Tree Depth

An algorithm can determine the deepest branch in a tree by recursively asking its child nodes how deep they are. The *depth* of a node is the number of edges between it and the root node. The root node itself has a depth of 0, the immediate child of the root node has a depth of 1, and so on. You may need this information as part of a larger algorithm or to gather information about the general size of the tree data structure.

We can have a function named `getDepth()` take a node for an argument and return the depth of its deepest child node. A leaf node (the base case) simply returns 0.

For example, given the root node of the tree in [Figure 4-1](#), we could call `getDepth()` and pass it the root node (the A node). This would return the depth of its children, the B and C nodes, plus one. The function must make a recursive call to `getDepth()` to find out this information. Eventually, the A node would call `getDepth()` on C, which would call it on E. When E calls `getDepth()` with its two children, G and H, they both return 0, so `getDepth()` called on E returns 1, making `getDepth()` called on C return 2, and making `getDepth()` called on A (the root node) return 3. Our tree's greatest depth is three levels.

Let's ask our three recursive algorithm questions for the `getDepth()` function:

What is the base case? A leaf node with no children, which by its nature has a depth of one level.

What argument is passed to the recursive function call? The node whose greatest depth we want to find.

How does this argument become closer to the base case? A DAG has no cycles, so following the descendant nodes will eventually reach a leaf node.

The following `getDepth.py` program contains a recursive `getDepth()` function that returns the number of levels contained in the deepest node in the tree: